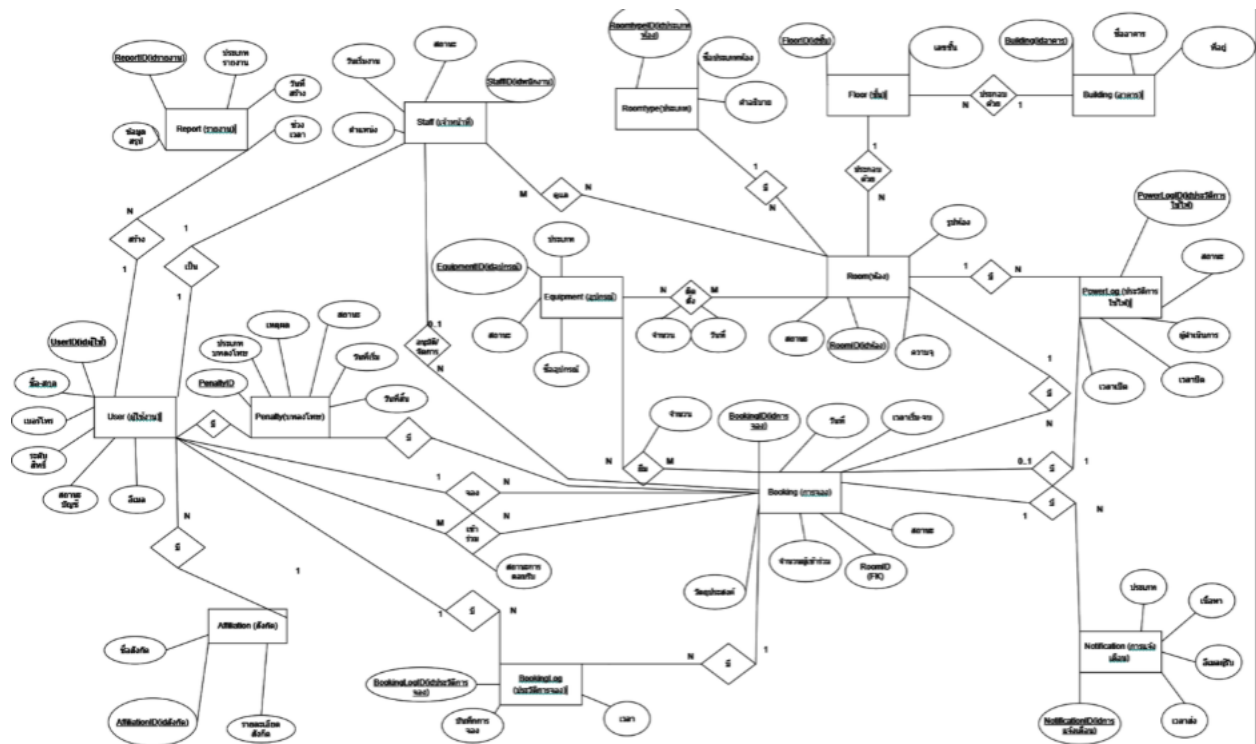


Term Project: แก้ไข Database Schema สู่ระดับ Production

ชื่อ-สกุล / กลุ่ม (Group Name)	เบส888
สมาชิก รหัสนักศึกษา ชื่อ สามสกุล	1.นางสาว พุทธิดา ชนะโยธา เลขรหัสนักศึกษา 683380139-4 2.นางสาว ศิริพรรณ แก้วกนก เลขรหัสนักศึกษา 683380636-0 3.นาย รัชสิริ กิรติเรขา เลขรหัสนักศึกษา 683380384-1 4.นาย กิตติภณ สุภัญญา เลขรหัสนักศึกษา 683380357-4 5.นาย เขมวิชญ์ สารพิชญ์ เลขรหัสนักศึกษา 683380401-7 6.นาย กชกรณ ไชยสาร เลขรหัสนักศึกษา 683380353-2 7.นาย ปราโมทย์ ชำกรม เลขรหัสนักศึกษา 683380630-2
วันที่ส่งงาน	4/10/2026
หัวข้อโครงการ	ระบบจองห้องเรียน/ห้องประชุม

1.) ฐานข้อมูลเดิม

1.1) ER Diagram ฉบับเต็ม



1.2) รายการปัญหาที่พบในฐานข้อมูลเดิมและผลกระทบ

1.2.1) ด้าน Normalization และ Data Integrity (การลดความซ้ำซ้อนและเงื่อนไขความถูกต้อง)

○ การไม่มี Unique Constraints ป้องกันการจองซ้อน (Overlapping Bookings) :

- **ปัญหาที่พบ** : ตาราง Booking มี PK คือ BookingID และกำหนด Candidate Key ไว้ในเอกสารเป็น (RoomID, วันที่, เวลาเริ่ม) แต่ในทางปฏิบัติช่วงเวลการจองเป็นช่วง (เวลาเริ่ม - เวลาจบ) การใช้ Unique Composite Key ปกติไม่สามารถป้องกันการจองเหลื่อมเวลาได้ (เช่น จอง 10:00-12:00 น. แล้วมีคนมาจอง 11:00-13:00 น.)

- **ผลกระทบ** : เสี่ยงต่อการเกิด Anomaly (ความผิดปกติของข้อมูล) ข้อมูลการจองทับซ้อนกันในห้องเดียวกัน ช่วงเวลาเดียวกัน ระบบไม่สามารถรันดี Business Rule ข้อ 4(ห้องเดียวกันห้ามมีการจองซ้อนช่วงเวลาเกิน) ได้ที่ระดับ Database

○ ขาด Check Constraint สำหรับช่วงเวลา และจำนวน :

- **ปัญหาที่พบ** : ตาราง Booking และ PowerLog ไม่มีการระบุ Check Constraint ให้ เวลาจบ > เวลาเริ่ม หรือตาราง BookingEquipment / RoomEquipment ไม่ได้ระบุ จำนวน > 0

- **ผลกระทบ** : เกิดข้อมูลที่ผิดพลาดเชิงตรรกะ เช่น เวลาเลิกจองน้อยกว่าเวลาเริ่มจอง หรือยืมอุปกรณ์จำนวนติดลบ

○ ขาดการระบุ Referential Integrity Behavior (ON DELETE) :

- **ปัญหาที่พบ** : Foreign Keys (FK) ทั้งหมดใน Schema ไม่ได้ระบุเงื่อนไข ON DELETE

(เช่น CASCADE, SET NULL หรือค่าปริยายคือไม่อนุญาตให้ลบ)

- **ผลกระทบ** : เมื่อมีการลบข้อมูลหลัก เช่น ลบ User หรือ Room อาจเกิดปัญหา Orphan Records (ข้อมูลเด็กค้าง) หรือระบบ Error จาก Database Foreign Key Constraint Failure

○ ขาด Not Null Constraint ใน Foreign Keys และคอลัมน์สำคัญ

-**ปัญหาที่พบ**: คอลัมน์สำคัญเช่น users_user_id, room_room_id ในตาราง booking หรือ building_building_id ในตาราง floor เดิมอนุญาตให้เป็น NULL ได้

-การแก้ไขใน DDL: เพิ่ม NOT NULL ใน DDL ใหม่เพื่การันตีว่าการจองต้องระบุผู้จองและห้องเสมอ

1.2.2) ด้าน Data Types (ความเหมาะสมของชนิดข้อมูล)

○ ชนิดข้อมูลวัน-เวลา แยกคอลัมน์และไม่ระบุ Timezone :

- ปัญหาที่พบ : ตาราง Booking แยกเก็บ วันที่, เวลาเริ่ม, เวลาจบ ส่วนตารางอื่นๆ เก็บเวลาแบบ Text หรือ DateTime ทั่วไปโดยไม่ระบุ Timezone
- ผลกระทบ : คำนวณช่วงเวลาหรือเปรียบเทียบข้ามวันได้ยาก ซับซ้อนในการเขียน Query และเสี่ยงต่อปัญหาความคลาดเคลื่อนของเวลาเมื่อเปลี่ยน Server หรือมี User ต่าง Timezone

○ ชนิดข้อมูลของสถานะและตัวเลือกที่แน่นอน (Status Flags) :

- ปัญหาที่พบ : สถานะบัญชี, สถานะห้อง, สถานะการจอง, สถานะบทลงโทษ เก็บเป็นข้อความทั่วไป (Text/String) โดยไม่มีการจำกัดค่า (VARCHAR โดยไม่มี ENUM หรือ CHECK)
- ผลกระทบ : ผู้ใช้หรือแอปพลิเคชันอาจสะกดคำผิด เช่น "Approved", "approved", "อนุมัติ" ทำให้การค้นหาและ Query รายงานผิดพลาด

1.2.3) ด้าน Business Rule และความครบถ้วนของ Schema (Missing Entities/Fields)

ขาดข้อมูลเวลาและเวลาอนุมัติ (Approval Audit)- ปัญหาที่พบ :
ตาราง Booking ไม่ได้ระบุคอลัมน์เก็บว่า Staff

คนไหนเป็นผู้อนุมัติ/ปฏิเสธ และอนุมัติเมื่อใด มีเพียง Relationship ที่ระบุว่า Staff อนุมัติ Booking

- **ผลกระทบ :** ไม่สามารถตรวจสอบย้อนหลัง (Audit)

ได้ว่าการจองรายการนี้ถูกอนุมัติหรือปฏิเสธ และอนุมัติเวลาไหน

ขาดตารางบันทึกวันหยุดประจำปี (Holiday Entity)

-**ปัญหาที่พบ:** Schema เดิมไม่มีตารางเก็บข้อมูลวันหยุด

-**การแก้ไขใน DDL:** เพิ่มตาราง holiday เพื่อให้ระบบนำไปใช้ตรวจสอบป้องกันการจองห้องในวันหยุดได้

1.2.4) ด้าน User, Authentication & Audit Trail (ผู้ใช้ สิทธิ์ และการติดตาม)

ขาดคอลัมน์ติดตามเวลาและการทำ Soft Delete (Audit Fields / Soft Delete):

- **ปัญหาที่พบ:** ตารางหลักขาดคอลัมน์บันทึกเวลาสร้าง/แก้ไข (created_at, updated_at) และไม่มีกลไก Soft Delete (is_deleted, deleted_at)
- **ผลกระทบ:** ไม่สามารถติดตามได้ว่าข้อมูลถูกสร้างหรืออัปเดตเมื่อใด และการลบข้อมูลเป็นการ Hard Delete ทำให้ประวัติการจองในอดีตสูญหาย

1.2.5) ด้าน Performance and Indexing (ประสิทธิภาพและการค้นหา)

- ขาดการกำหนด Index รองรับ Query หลัก :

- **ปัญหาที่พบ :** ขาด Index สำหรับคอลัมน์ที่ถูกค้นหบ่อย เช่น Booking (RoomID, เวลาเริ่ม, เวลาจบ) สำหรับเช็กห้องว่าง และ Booking (UserID, วันที่สร้าง) สำหรับดูประวัติการจองของผู้ใช้

- **ผลกระทบ :** เมื่อปริมาณข้อมูลระดับ Production เพิ่มขึ้น การค้นหาห้องว่างหรือดูประวัติการจองจะช้าลงอย่างมากเนื่องจากต้องทำ Full Table Scan

1.2.6) ด้าน IoT Integration & High Volume Data (ระบบตัดไฟและ Log)

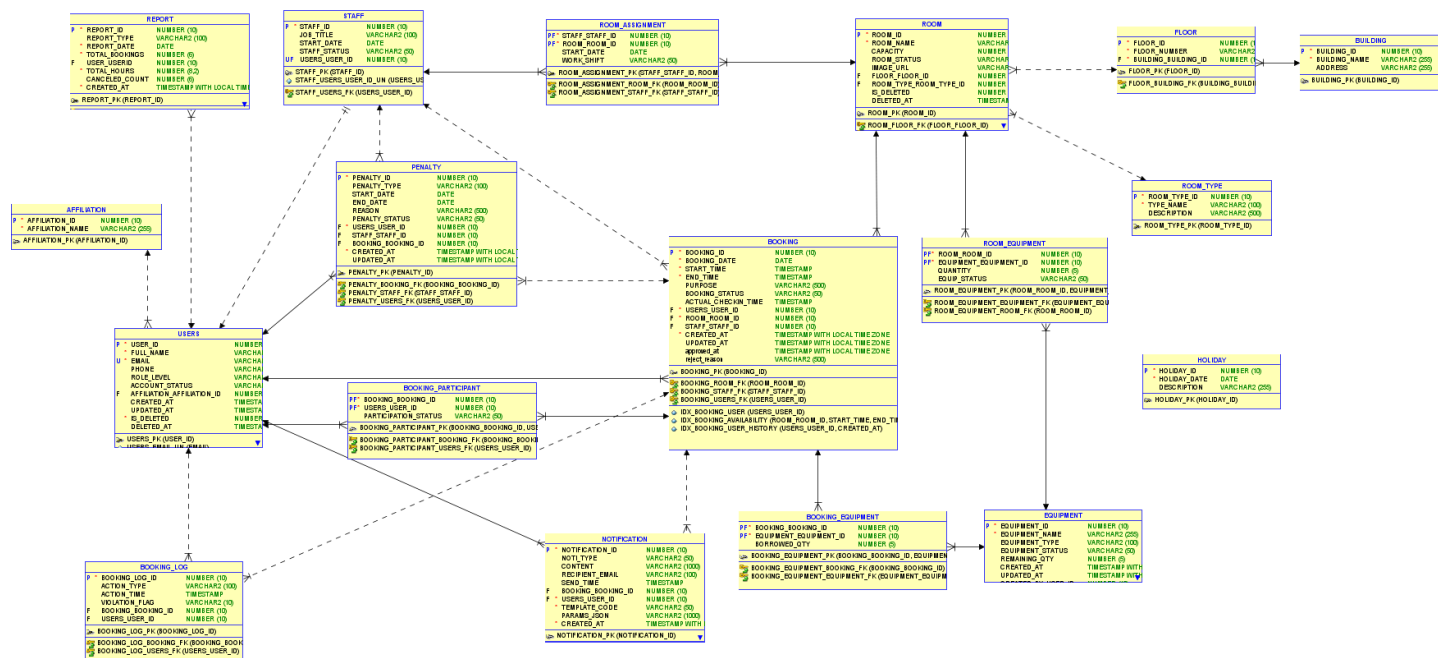
- ตาราง PowerLog ไม่ถูกแยกโครงสร้างเพื่อรองรับข้อมูลขนาดใหญ่ (Time-series Data) :

- ปัญหาที่พบ : บันทึกประวัติการใช้ไฟ/ตัดไฟลูกออกแบบเป็นตาราง Relational ทำให้รวมกับฐานข้อมูลหลัก

- ผลกระทบ : ระบบ IoT ส่งสัญญาณและบันทึกสถานะบ่อยครั้ง การบันทึกในตารางปกติโดยไม่มี Partitioning ทำให้ตารางโตอย่างรวดเร็ว ส่งผลกระทบต่อ Performance รวมของฐานข้อมูล Production

2.) ฐานข้อมูลใหม่

2.1) ER Diagram ฉบับใหม่



2.2) DDL Script

```
CREATE TABLE affiliation (affiliation_id NUMBER(10) NOT NULL,
affiliation_name VARCHAR2(255) NOT NULL);
ALTER TABLE affiliation ADD CONSTRAINT affiliation_pk
PRIMARY KEY (affiliation_id);
CREATE TABLE booking (booking_id NUMBER(10) NOT NULL,
booking_date DATE NOT NULL, start_time TIMESTAMP NOT
NULL, end_time TIMESTAMP NOT NULL, purpose
VARCHAR2(500), booking_status VARCHAR2(50),
```

```
actual_checkin_time TIMESTAMP, users_user_id NUMBER(10)
NOT NULL, room_room_id NUMBER(10) NOT NULL, staff_staff_id
NUMBER(10), created_at TIMESTAMP WITH LOCAL TIME ZONE
DEFAULT current_timestamp NOT NULL, updated_at
TIMESTAMP WITH LOCAL TIME ZONE DEFAULT
current_timestamp, approved_at TIMESTAMP WITH LOCAL TIME
ZONE, reject_reason VARCHAR2(500));
CREATE INDEX idx_booking_user ON booking (users_user_id);
CREATE INDEX idx_booking_availability ON booking
(room_room_id, start_time, end_time);
CREATE INDEX idx_booking_user_history ON booking
(users_user_id, created_at);
CREATE SEQUENCE booking_log_seq START WITH 1
INCREMENT BY 1;
ALTER TABLE booking ADD CONSTRAINT ck_booking_time
CHECK (end_time > start_time);
ALTER TABLE booking ADD CONSTRAINT ck_booking_status
CHECK (booking_status IN ('PENDING', 'APPROVED', 'REJECTED',
'CANCELLED', 'CHECKED_IN', 'COMPLETED'));
ALTER TABLE booking ADD CONSTRAINT booking_pk PRIMARY
KEY (booking_id);
CREATE TABLE booking_equipment (booking_booking_id
NUMBER(10) NOT NULL, equipment_equipment_id NUMBER(10)
NOT NULL, borrowed_qty NUMBER(5));
ALTER TABLE booking_equipment ADD CONSTRAINT ck_be_qty
CHECK (borrowed_qty > 0);
ALTER TABLE booking_equipment ADD CONSTRAINT
booking_equipment_pk PRIMARY KEY (booking_booking_id,
equipment_equipment_id);
```

```
CREATE TABLE booking_log (booking_log_id NUMBER(10) NOT
NULL, action_type VARCHAR2(100), action_time TIMESTAMP,
violation_flag VARCHAR2(10), booking_booking_id NUMBER(10),
users_user_id NUMBER(10));
ALTER TABLE booking_log ADD CONSTRAINT
booking_log_violation_ck CHECK (violation_flag IN ('YES', 'NO'));
ALTER TABLE booking_log ADD CONSTRAINT booking_log_pk
PRIMARY KEY (booking_log_id);
CREATE TABLE booking_participant (booking_booking_id
NUMBER(10) NOT NULL, users_user_id NUMBER(10) NOT NULL,
participation_status VARCHAR2(50));
ALTER TABLE booking_participant ADD CONSTRAINT
booking_participant_pk PRIMARY KEY (booking_booking_id,
users_user_id);
CREATE TABLE building (building_id NUMBER(10) NOT NULL,
building_name VARCHAR2(255) NOT NULL, address
VARCHAR2(255));
ALTER TABLE building ADD CONSTRAINT building_pk PRIMARY
KEY (building_id);
CREATE TABLE equipment (equipment_id NUMBER(10) NOT
NULL, equipment_name VARCHAR2(255) NOT NULL,
equipment_type VARCHAR2(100), equipment_status
VARCHAR2(50), remaining_qty NUMBER(5), created_at
TIMESTAMP WITH LOCAL TIME ZONE DEFAULT
current_timestamp, updated_at TIMESTAMP WITH LOCAL TIME
ZONE, created_by_user_id NUMBER(10), is_deleted NUMBER(1)
DEFAULT 0, deleted_at TIMESTAMP WITH LOCAL TIME ZONE);
ALTER TABLE equipment ADD CONSTRAINT equipment_qty_ck
CHECK (remaining_qty >= 0);
```

```
ALTER TABLE equipment ADD CONSTRAINT equipment_pk  
PRIMARY KEY (equipment_id);  
CREATE TABLE floor (floor_id NUMBER(10) NOT NULL,  
floor_number VARCHAR2(50) NOT NULL, building_building_id  
NUMBER(10) NOT NULL);  
ALTER TABLE floor ADD CONSTRAINT floor_pk PRIMARY KEY  
(floor_id);  
CREATE TABLE holiday (holiday_id NUMBER(10) NOT NULL,  
holiday_date DATE NOT NULL, description VARCHAR2(255));  
ALTER TABLE holiday ADD CONSTRAINT holiday_pk PRIMARY  
KEY (holiday_id);  
CREATE TABLE notification (notification_id NUMBER(10) NOT  
NULL, noti_type VARCHAR2(50), content VARCHAR2(1000),  
recipient_email VARCHAR2(100), send_time TIMESTAMP,  
booking_booking_id NUMBER(10), users_user_id NUMBER(10)  
NOT NULL, template_code VARCHAR2(50) NOT NULL,  
params_json VARCHAR2(1000), created_at TIMESTAMP WITH  
LOCAL TIME ZONE DEFAULT current_timestamp NOT NULL);  
CREATE SEQUENCE notification_seq START WITH 1 INCREMENT  
BY 1;  
ALTER TABLE notification ADD CONSTRAINT notification_pk  
PRIMARY KEY (notification_id);  
CREATE TABLE penalty (penalty_id NUMBER(10) NOT NULL,  
penalty_type VARCHAR2(100), start_date DATE, end_date DATE,  
reason VARCHAR2(500), penalty_status VARCHAR2(50),  
users_user_id NUMBER(10) NOT NULL, staff_staff_id  
NUMBER(10), booking_booking_id NUMBER(10), created_at  
TIMESTAMP WITH LOCAL TIME ZONE DEFAULT  
current_timestamp NOT NULL, updated_at TIMESTAMP WITH  
LOCAL TIME ZONE DEFAULT current_timestamp);
```

```
ALTER TABLE penalty ADD CONSTRAINT penalty_date_ck CHECK
(end_date >= start_date);
ALTER TABLE penalty ADD CONSTRAINT penalty_pk PRIMARY
KEY (penalty_id);
CREATE TABLE report (report_id NUMBER(10) NOT NULL,
report_type VARCHAR2(100), report_date DATE NOT NULL,
total_bookings NUMBER(6) NOT NULL, user_userid NUMBER(10),
total_hours NUMBER(8, 2) NOT NULL, canceled_count
NUMBER(6), created_at TIMESTAMP WITH LOCAL TIME ZONE
DEFAULT current_timestamp NOT NULL);
ALTER TABLE report ADD CONSTRAINT report_pk PRIMARY KEY
(report_id);
CREATE TABLE room (room_id NUMBER(10) NOT NULL,
room_name VARCHAR2(100) NOT NULL, capacity NUMBER(5),
room_status VARCHAR2(50), image_url VARCHAR2(500),
floor_floor_id NUMBER(10), room_type_room_type_id
NUMBER(10), is_deleted NUMBER(1) DEFAULT 0, deleted_at
TIMESTAMP WITH LOCAL TIME ZONE);
ALTER TABLE room ADD CONSTRAINT room_pk PRIMARY KEY
(room_id);
CREATE TABLE room_assignment (staff_staff_id NUMBER(10)
NOT NULL, room_room_id NUMBER(10) NOT NULL, start_date
DATE, work_shift VARCHAR2(50));
ALTER TABLE room_assignment ADD CONSTRAINT
room_assignment_pk PRIMARY KEY (staff_staff_id,
room_room_id);
CREATE TABLE room_equipment (room_room_id NUMBER(10)
NOT NULL, equipment_equipment_id NUMBER(10) NOT NULL,
quantity NUMBER(5), equip_status VARCHAR2(50));
```

```
ALTER TABLE room_equipment ADD CONSTRAINT
room_equipment_qty_ck CHECK (quantity > 0);
ALTER TABLE room_equipment ADD CONSTRAINT
room_equipment_pk PRIMARY KEY (room_room_id,
equipment_equipment_id);
CREATE TABLE room_type (room_type_id NUMBER(10) NOT
NULL, type_name VARCHAR2(100) NOT NULL, description
VARCHAR2(500));
ALTER TABLE room_type ADD CONSTRAINT room_type_pk
PRIMARY KEY (room_type_id);
CREATE TABLE staff (staff_id NUMBER(10) NOT NULL, job_title
VARCHAR2(100), start_date DATE, staff_status VARCHAR2(50),
users_user_id NUMBER(10));
ALTER TABLE staff ADD CONSTRAINT staff_pk PRIMARY KEY
(staff_id);
ALTER TABLE staff ADD CONSTRAINT staff_users_user_id_un
UNIQUE (users_user_id);
CREATE TABLE users (user_id NUMBER(10) NOT NULL, full_name
VARCHAR2(255) NOT NULL, email VARCHAR2(100) NOT NULL,
phone VARCHAR2(20), role_level VARCHAR2(50), account_status
VARCHAR2(50), affiliation_affiliation_id NUMBER(10), created_at
TIMESTAMP WITH LOCAL TIME ZONE DEFAULT
current_timestamp, updated_at TIMESTAMP WITH LOCAL TIME
ZONE DEFAULT current_timestamp, is_deleted NUMBER(1)
DEFAULT 0 NOT NULL, deleted_at TIMESTAMP WITH LOCAL
TIME ZONE);
ALTER TABLE users ADD CONSTRAINT users_pk PRIMARY KEY
(user_id);
ALTER TABLE users ADD CONSTRAINT users_email_un UNIQUE
(email);
```

```
ALTER TABLE booking_equipment ADD CONSTRAINT
booking_equipment_booking_fk FOREIGN KEY
(booking_booking_id) REFERENCES booking (booking_id);
ALTER TABLE booking_equipment ADD CONSTRAINT
booking_equipment_equipment_fk FOREIGN KEY
(equipment_equipment_id) REFERENCES equipment
(equipment_id);
ALTER TABLE booking_log ADD CONSTRAINT
booking_log_booking_fk FOREIGN KEY (booking_booking_id)
REFERENCES booking (booking_id);
ALTER TABLE booking_log ADD CONSTRAINT
booking_log_users_fk FOREIGN KEY (users_user_id) REFERENCES
users (user_id);
ALTER TABLE booking_participant ADD CONSTRAINT
booking_participant_booking_fk FOREIGN KEY
(booking_booking_id) REFERENCES booking (booking_id);
ALTER TABLE booking_participant ADD CONSTRAINT
booking_participant_users_fk FOREIGN KEY (users_user_id)
REFERENCES users (user_id);
ALTER TABLE booking ADD CONSTRAINT booking_room_fk
FOREIGN KEY (room_room_id) REFERENCES room (room_id);
ALTER TABLE booking ADD CONSTRAINT booking_staff_fk
FOREIGN KEY (staff_staff_id) REFERENCES staff (staff_id);
ALTER TABLE booking ADD CONSTRAINT booking_users_fk
FOREIGN KEY (users_user_id) REFERENCES users (user_id);
ALTER TABLE floor ADD CONSTRAINT floor_building_fk
FOREIGN KEY (building_building_id) REFERENCES building
(building_id);
```

```
ALTER TABLE notification ADD CONSTRAINT
notification_booking_fk FOREIGN KEY (booking_booking_id)
REFERENCES booking (booking_id);
ALTER TABLE notification ADD CONSTRAINT
notification_users_fk FOREIGN KEY (users_user_id) REFERENCES
users (user_id);
ALTER TABLE penalty ADD CONSTRAINT penalty_booking_fk
FOREIGN KEY (booking_booking_id) REFERENCES booking
(booking_id);
ALTER TABLE penalty ADD CONSTRAINT penalty_staff_fk
FOREIGN KEY (staff_staff_id) REFERENCES staff (staff_id);
ALTER TABLE penalty ADD CONSTRAINT penalty_users_fk
FOREIGN KEY (users_user_id) REFERENCES users (user_id);
ALTER TABLE report ADD CONSTRAINT report_users_fk
FOREIGN KEY (user_userid) REFERENCES users (user_id);
ALTER TABLE room_assignment ADD CONSTRAINT
room_assignment_room_fk FOREIGN KEY (room_room_id)
REFERENCES room (room_id);
ALTER TABLE room_assignment ADD CONSTRAINT
room_assignment_staff_fk FOREIGN KEY (staff_staff_id)
REFERENCES staff (staff_id);
ALTER TABLE room_equipment ADD CONSTRAINT
room_equipment_equipment_fk FOREIGN KEY
(equipment_equipment_id) REFERENCES equipment
(equipment_id);
ALTER TABLE room_equipment ADD CONSTRAINT
room_equipment_room_fk FOREIGN KEY (room_room_id)
REFERENCES room (room_id);
ALTER TABLE room ADD CONSTRAINT room_floor_fk FOREIGN
KEY (floor_floor_id) REFERENCES floor (floor_id);
```

```

ALTER TABLE room ADD CONSTRAINT room_room_type_fk
FOREIGN KEY (room_type_room_type_id) REFERENCES
room_type (room_type_id);
ALTER TABLE staff ADD CONSTRAINT staff_users_fk FOREIGN
KEY (users_user_id) REFERENCES users (user_id);
ALTER TABLE users ADD CONSTRAINT users_affiliation_fk
FOREIGN KEY (affiliation_affiliation_id) REFERENCES affiliation
(affiliation_id);

```

3.) จุดที่แตกต่าง และ เพิ่มเข้ามา

รายละเอียดการเปลี่ยนแปลง

ลำดับ	ประเภท	ตาราง / คอลัมน์	เดิม	ใหม่	เหตุผล
1	เพิ่ม	holiday	ไม่มี	ตารางใหม่ (holiday_id, holiday_date, description)	บันทึกวันหยุดประจำปี เพื่อให้ระบบตรวจสอบป้องกันการ เปิดจองห้องในวันหยุด
2	แก้ไข	booking.start_time, booking.end_time	แยกเก็บ วันที่ (DATE), เวลาเริ่ม (TIME), เวลาจบ (TIME)	start_time TIMESTAMP, end_time TIMESTAMP	รวมเป็น TIMESTAMP เพื่อรองรับการจองข้ามวัน และคำนวณช่วงเวลาแบบ Date-Time ได้ถูกต้องแม่นยำ
3	เพิ่ม	booking (Audit Fields)	ไม่มี	created_at TIMESTAMP WITH LOCAL TIME ZONE, updated_at TIMESTAMP WITH LOCAL TIME ZONE	ติดตามและตรวจสอบย้อนหลัง ว่าข้อมูลการจองถูกสร้างและ

					แก้ไขเมื่อใด
4	เพิ่ม	booking.actual_checkin_time	ไม่มี	actual_checkin_time TIMESTAMP	บันทึกเวลาที่ผู้ใช้เข้าใช้งานห้องจริง เพื่อตรวจสอบการไม่เข้าใช้งาน (No-show) และการลงโทษ
5	เพิ่ม	users (Soft Delete & Audit)	มีเพียงข้อมูลพื้นฐาน	created_at, updated_at, is_deleted NUMBER(1), deleted_at TIMESTAMP	รองรับ Soft Delete ไม่ให้ข้อมูล ผู้ใช้และประวัติการจองหายไป เมื่อมีการลบบัญชี
6	เพิ่ม	room (Soft Delete)	ไม่มี	is_deleted NUMBER(1), deleted_at TIMESTAMP	เพื่อซ่อนห้องที่ปิดปรับปรุงหรือ ยกเลิกการใช้งาน โดยไม่กระทบต่อประวัติการจองเดิม
7	เพิ่ม	equipment (Audit & Soft Delete)	มีเฉพาะข้อมูลอุปกรณ์	created_at, updated_at, created_by_user_id, is_deleted, deleted_at	ติดตามการลงทะเบียน/แก้ไข อุปกรณ์ และรองรับ Soft Delete เมื่ออุปกรณ์ชำรุดหรือ จำหน่ายออก
8	แก้ไข	booking_log	BookingLogID, BookingID, UserID, บันทึกการจอง, เวลา	booking_log_id, action_type, action_time, violation_flag, booking_booking_id, users_user_id	ปรับปรุงโครงสร้าง Log ให้ระบุ ประเภทการกระทำ (action_type) และแฟล็กการทำผิดเงื่อนไข (violation_flag) ได้ชัดเจนขึ้น
9	แก้ไข	notification	NotificationID, BookingID, ประเภท, เนื้อหา, อีเมลผู้รับ, เวลาที่ส่ง	เพิ่ม users_user_id, template_code, params_json, created_at	เปลี่ยนไปใช้ระบบ Template และ Dynamic JSON parameters เพื่อความยืดหยุ่นใน การส่งข้อความแจ้งเตือน

10	เพิ่ม	penalty (Audit Fields)	ไม่มี Audit Fields	created_at TIMESTAMP WITH LOCAL TIME ZONE, updated_at TIMESTAMP WITH LOCAL TIME ZONE	บันทึกเวลาที่มีการออกบทลงโทษ และการอัปเดตสถานะการลงโทษ
11	เพิ่ม	report (คอแลมน์สรุป)	มีเฉพาะฟิลด์สรุปทั่วไป	total_hours NUMBER(8,2), canceled_count NUMBER(6), created_at	เพิ่มฟิลด์คำนวณชั่วโมงรวม การใช้งาน และจำนวนครั้งที่ยกเลิก การจองเพื่อสร้างรายงานระดับ Executive
12	เพิ่ม	booking (Performance Indexes)	ไม่มี Index	idx_booking_availability (room_id, start_time, end_time) และ idx_booking_user_history (user_id, created_at) idx_booking_user (user_id)	เพิ่ม Index เพื่อเร่งความเร็วในการค้นหา ห้องว่างตามช่วงเวลา และค้นหาประวัติการจองของผู้ใช้ (แก้ปัญหา 1.2.5)
13	ลบ	PowerLog	มีตาราง PowerLog	ลบออกจาก Relational DB Schema หลัก	แยกข้อมูล IoT Time-series Log ออกจาก Relational DB หลัก เพื่อป้องกันปัญหาตารางโตเกินไป และ Performance ตกในระดับ Production
14	เพิ่ม	booking_log_seq, notification_seq	ไม่มี	CREATE SEQUENCE ทั้ง 2 ตัว	ใช้สร้างรหัส log และการแจ้งเตือนใน Transaction ถ้าไม่มีจะรันไม่ผ่าน (ORA- 02289)
15	แก้ไข	booking (users_user_id, room_room_id, start_time, end_time)และ	เว้นได้	NOT NULL	การจองต้องมีผู้จอง ห้อง และช่วงเวลาเสมอ บทลงโทษต้องระบุผู้ถูกลงโทษ และชั้นต้องอยู่ในอาคาร

		penalty (users_user_id), floor (building_building_id)			
16	เพิ่ม	booking, booking_equipment, room_equipment, equipment, penalty, booking_lo	ไม่มีเช็ค	CHECK เวลาจบ > เวลาเริ่ม, จำนวน > 0, วันที่สิ้นสุด >= วันที่เริ่ม, ค่าสถานะการจอง และ violation_flag	แก้ปัญหา 1.2.1 (Check Constraint ช่วงเวลาและจำนวน) และ 1.2.2 (Status Flags ของการจอง) ป้องกันข้อมูลผิดตรรกะ และสถานะสะกดผิด
17	เพิ่ม	booking.approved_at, reject_reason	ไม่มี	approved_at TIMESTAMP WITH LOCAL TIME ZONE, reject_reason VARCHAR2(500)	ตรวจสอบย้อนหลังได้ว่าเจ้าหน้าที่อนุมัติหรือ ปฏิเสธเมื่อใดและเพราะอะไร

4.) Transaction ภายในโครงการของตนเอง

4.1) ชื่อและคำอธิบายของ Transaction 1

Transaction ที่ 1 : กระบวนการส่งคำจองห้องเรียน/ห้องประชุม พร้อมยืมอุปกรณ์ (Create Booking with Equipment)

คำอธิบายเมื่อผู้ใช้ส่งคำจองห้อง ระบบต้องตรวจสอบความพร้อมใช้งาน ตัดสต็อกจำนวนอุปกรณ์ที่ยืม บันทึกข้อมูลการจองลงตาราง BOOKING
บันทึกอุปกรณ์ที่ยืมลงตาราง BOOKING_EQUIPMENT และบันทึกประวัติการทำรายการลงตาราง BOOKING_LOG

ตารางที่ถูกแก้ไข: BOOKING, BOOKING_EQUIPMENT, BOOKING_LOG

เงื่อนไขที่ต้อง Rollback : พบการจองซ้อนทับในช่วงเวลาเดียวกัน ($v_overlap > 0$) , จำนวนอุปกรณ์คงเหลือไม่เพียงพอ ($v_avail_qty < v_req_qty$)

เกิดข้อผิดพลาดทางเทคนิค เช่น Foreign Key Violation หรือ Database Connection ขัดข้องขณะ Execute เหตุผลที่ต้องอยู่ใน Transaction เดียวกัน: เพื่อรักษาคุณสมบัติ Atomicity หากตัดสต็อกอุปกรณ์สำเร็จ แต่เพิ่มข้อมูลการจองลงตาราง BOOKING ไม่สำเร็จ จะทำให้สต็อกอุปกรณ์คลาดเคลื่อน หรือหากจองสำเร็จแต่บันทึกยืมอุปกรณ์ไม่สำเร็จ ข้อมูลการจองจะไม่สมบูรณ์ตาม Business Context

4.2) SQL Script 1

```
DECLARE
v_booking_id NUMBER := 1001;
v_user_id NUMBER := 50;
v_room_id NUMBER := 5;
v_equip_id NUMBER := 12;
v_req_qty NUMBER := 2;
v_start_time TIMESTAMP := TO_TIMESTAMP('2026-10-10 09:00:00', 'YYYY-MM-DD HH24:MI:SS');
v_end_time TIMESTAMP := TO_TIMESTAMP('2026-10-10 12:00:00', 'YYYY-MM-DD HH24:MI:SS');
v_avail_qty NUMBER;
v_overlap NUMBER;
v_dummy NUMBER;
BEGIN
-- ล็อกแถวห้องก่อน กันคนจองห้องเดียวกันพร้อมกัน (race condition)
SELECT room_id INTO v_dummy
FROM room
WHERE room_id = v_room_id FOR UPDATE;

-- ตรวจสอบการจองซ้อนช่วงเวลา (รวมห้องที่มีคนเช็คอินใช้อยู่)
SELECT COUNT(*) INTO v_overlap
FROM booking
WHERE room_id = v_room_id
AND booking_status IN ('PENDING', 'APPROVED', 'CHECKED_IN')
AND (start_time < v_end_time AND end_time > v_start_time);

IF v_overlap > 0 THEN
```

```
RAISE_APPLICATION_ERROR(-20001, 'ห้องไม่ว่างในช่วงเวลาดังกล่าว');  
END IF;
```

```
SELECT remaining_qty INTO v_avail_qty  
FROM equipment  
WHERE equipment_id = v_equip_id FOR UPDATE;
```

```
IF v_avail_qty < v_req_qty THEN  
    RAISE_APPLICATION_ERROR(-20002, 'จำนวนอุปกรณ์ไม่พอสำหรับการยืม');  
END IF;
```

```
INSERT INTO booking (  
    booking_id, booking_date, start_time, end_time, purpose,  
    booking_status, users_user_id, room_room_id, created_at  
) VALUES (  
    v_booking_id, TRUNC(v_start_time), v_start_time, v_end_time, 'ประชุมโครงการ',  
    'PENDING', v_user_id, v_room_id, CURRENT_TIMESTAMP  
);
```

```
INSERT INTO booking_equipment (  
    booking_booking_id, equipment_equipment_id, borrowed_qty  
) VALUES (  
    v_booking_id, v_equip_id, v_req_qty  
);
```

```
-- ตัดจำนวนอุปกรณ์คงเหลือ  
UPDATE equipment  
SET remaining_qty = remaining_qty - v_req_qty,  
    updated_at = CURRENT_TIMESTAMP  
WHERE equipment_id = v_equip_id;
```

```
INSERT INTO booking_log (  
    booking_log_id, action_type, action_time, violation_flag, booking_booking_id, users_user_id  
) VALUES (  
    booking_log_seq.NEXTVAL, 'CREATE_BOOKING', CURRENT_TIMESTAMP, 'NO', v_booking_id, v_user_id
```

```
);

COMMIT;
DBMS_OUTPUT.PUT_LINE('สร้างคำจองสำเร็จ');
EXCEPTION
WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('เกิดข้อผิดพลาด ต้อง Rollback: ' || SQLERRM);
END;
/
```

4.3) ชื่อและคำอธิบายของ Transaction 2

Transaction ที่ 2 : กระบวนการอนุมัติการจองโดยเจ้าหน้าที่ (Staff Approval Workflow)

คำอธิบาย : เมื่อเจ้าหน้าที่กดอนุมัติคำจอง ระบบต้องทำการอัปเดตสถานะการจองในตาราง BOOKING ระบุ staff_id ผู้อนุมัติ สร้างข้อความแจ้งเตือนผู้ใช้ลงตาราง NOTIFICATION และบันทึกประวัติลงตาราง BOOKING_LOG

ตารางที่ถูกแก้ไข : BOOKING, NOTIFICATION, BOOKING_LOG

เงื่อนไขที่ต้อง Rollback :

รายการจองไม่ได้อยู่ในสถานะ 'PENDING' (ถูกยกเลิกหรืออนุมัติไปก่อนหน้านี้แล้ว) , ไม่พบข้อมูลผู้ใช้หรืออีเมลผู้รับการแจ้งเตือน, เกิด System Error ขณะบันทึก NOTIFICATION หรือ BOOKING_LOG

เหตุผลที่ต้องอยู่ใน Transaction เดียวกัน : เพื่อป้องกันปัญหา Data Inconsistency หากสถานะการจองถูกเปลี่ยนเป็น 'APPROVED' ในตาราง BOOKING แต่ระบบไม่สามารถบันทึก Queue การส่งอีเมลแจ้งเตือนลงตาราง NOTIFICATION ได้ จะทำให้ผู้ใช้ไม่ได้รับผลการอนุมัติ และไม่สามารถตรวจสอบประวัติย้อนหลังใน Log ได้ SQL Script

4.4) SQL Script 2

```
DECLARE

v_booking_id NUMBER := 1001;

v_staff_id NUMBER := 2;
```

```
v_user_id NUMBER;
```

```
v_staff_user_id NUMBER;
```

```
v_user_email VARCHAR2(100);
```

```
v_status VARCHAR2(50);
```

```
BEGIN
```

```
-- Step 1: ตรวจสอบสถานะการจองปัจจุบัน และ Lock Record
```

```
SELECT booking_status, users_user_id
```

```
INTO v_status, v_user_id
```

```
FROM booking
```

```
WHERE booking_id = v_booking_id
```

```
FOR UPDATE;
```

```
IF v_status != 'PENDING' THEN
```

```
RAISE_APPLICATION_ERROR(-20003, 'รายการจองนี้ไม่ได้อยู่ในสถานะรออนุมัติ');
```

```
END IF;
```

```
-- ดึงอีเมลผู้จอง (ถ้าไม่พบจะเกิด NO_DATA_FOUND และ Rollback)
```

```
SELECT email
```

```
INTO v_user_email
```

```
FROM users
```

```
WHERE user_id = v_user_id;
```

```
-- ดึงรหัสผู้ใช้ของเจ้าหน้าที่ที่อนุมัติ (ถ้าไม่พบจะเกิด NO_DATA_FOUND และ Rollback)
```

```
SELECT users_user_id
```

```
INTO v_staff_user_id
```

```
FROM staff
```

```
WHERE staff_id = v_staff_id;
```

```
-- Step 2: อัปเดตสถานะการจองเป็น APPROVED พร้อมผู้อนุมัติและเวลาอนุมัติ
```

```
UPDATE booking
```

```
SET booking_status = 'APPROVED',
```

```
staff_staff_id = v_staff_id,
```

```
approved_at = CURRENT_TIMESTAMP,
```

```
updated_at = CURRENT_TIMESTAMP
```

```
WHERE booking_id = v_booking_id;
```

-- Step 3: สร้าง Notification สำหรับส่งแจ้งเตือนผู้จอง

```
INSERT INTO notification (  
  
notification_id, noti_type, content, recipient_email, send_time,  
  
booking_booking_id, users_user_id, template_code, created_at  
  
) VALUES (  
  
notification_seq.NEXTVAL, 'EMAIL',  
  
'คำขอจองห้องได้รับการอนุมัติเรียบร้อยแล้ว',  
  
v_user_email, CURRENT_TIMESTAMP,  
  
v_booking_id, v_user_id, 'BOOKING_APPROVED', CURRENT_TIMESTAMP  
  
);
```

-- Step 4: บันทึก Booking Log (ในนามเจ้าหน้าที่ผู้อนุมัติ)

```
INSERT INTO booking_log (  
  
booking_log_id, action_type, action_time, violation_flag,  
  
booking_booking_id, users_user_id  
  
) VALUES (  
  
booking_log_seq.NEXTVAL, 'APPROVE_BOOKING', CURRENT_TIMESTAMP, 'NO',  
  
v_booking_id, v_staff_user_id
```

);

COMMIT;

DBMS_OUTPUT.PUT_LINE('อนุมัติการจองสำเร็จ');

EXCEPTION

WHEN OTHERS THEN

ROLLBACK;

DBMS_OUTPUT.PUT_LINE('เกิดข้อผิดพลาด ต้อง Rollback: ' || SQLERRM);

END;

/